POO COM JAVASCRIPT

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

JavaScript: A Linguagem da Web Viva

Diferente de outras linguagens onde o ambiente de execução é previsível e controlado, o JavaScript nasceu no caos – criado em 10 dias para rodar dentro de um navegador e reagir ao que o usuário faz em tempo real. Hoje ele é a única linguagem que roda nativamente no navegador, domina o backend com Node.js e ainda sustenta aplicações mobile e desktop.

O que torna o JavaScript único?

- Onipresença: É a única linguagem que vive ao mesmo tempo no frontend e no backend, permitindo que um único desenvolvedor construa sistemas completos do zero.
- Dinamismo: JavaScript é de tipagem dinâmica e altamente flexível – objetos podem ser criados e modificados em tempo de execução, oferece uma liberdade que poucas linguagens permitem.
- Assincronismo: O JavaScript foi construído para não travar. Através de callbacks, Promises e async/await, ele lida com operações demoradas sem parar a execução –essencial para aplicações web modernas

INTRODUÇÃO

"Entender JavaScript é aceitar que a linguagem foi feita para o imprevisível. Aqui, mais do que seguir regras, você aprende a domar o comportamento de sistemas vivos."

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

RELACIONAMENTOS

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

Ou seja:

- 🔸 Composição → dependência total
- 🔹 Agregação → ligação fraca (referência)

CONSTRUTORES E COLECTIONS

1. JavaScript possui diversas estruturas para armazenar objetos:
   - Array, Map, Set

   E métodos para manipular coleções:
   - .forEach(), .map(), .filter(), .find()

INTERFACES

Interfaces em JavaScript são simuladas via classes abstratas

- JavaScript não tem interfaces formais nem classes abstratas nativas.
- O contrato é criado lançando um erro no método base com throw new Error().
- Classes filhas herdam e sobrescrevem o método – se não sobrescreverem, o erro aparece em tempo de execução.

EXERCÍCIO 1

Cenário: No mundo de Pokémon, treinadores levam criaturas chamadas Pokémon para batalhas em ginásios. Cada Pokémon pertence a uma espécie, tem um tipo, que define os seus golpes, e uma quantidade de vida. A vida não pode ser alterada diretamente de fora: só os golpes recebidos em combate podem reduzi-la, e ela nunca fica abaixo de zero.

Tarefa:

1. Crie a classe Pokemon, cujo constructor recebe apelido, tipo e vidaInicial:
   - apelido e tipo são atributos públicos;
   - a vida fica no atributo privado #vida, que começa com o valor de vidaInicial;
   - crie o getter vida (com get) e nenhum setter: de fora, a vida só pode ser lida;
   - crie o método sofrerDano(pontos), que subtrai os pontos da vida sem deixá-la negativa (se passar de zero, ela fica em 0) e imprime o apelido e a vida restante;
   - crie o método atacar(alvo), que recebe outro Pokémon, imprime um ataque genérico e chama alvo.sofrerDano(10).
2. Crie a classe filha Squirtle, cujo constructor recebe apelido e vidaInicial e chama super(...) passando o tipo fixo 'Água'. Sobrescreva atacar(alvo) para imprimir um golpe de água à sua escolha e chamar alvo.sofrerDano(20).

EXERCÍCIO 1

3. Crie a classe filha Bulbasaur, do mesmo jeito, com o tipo fixo 'Planta'. Sobrescreva atacar(alvo) para imprimir um golpe de planta à sua escolha e chamar alvo.sofrerDano(15).
4. No main, crie um Squirtle e um Bulbasaur, com apelidos à sua escolha. A vida inicial do Squirtle também é à sua escolha, mas o Bulbasaur deve começar com 30. Faça o Bulbasaur atacar o Squirtle uma vez e o Squirtle atacar o Bulbasaur duas vezes. Depois, imprima a vida dos dois usando o getter.
5. Tente imprimir o #vida de um dos Pokémon diretamente no main e observe o erro: o programa nem chega a rodar. Depois, transforme essa linha em comentário (//) para o programa voltar a rodar.

EXERCÍCIO 2

Cenário: Valorant é um jogo de tiro tático em que dois times de cinco **agentes** se enfrentam. Os agentes fazem parte do **Protocolo VALORANT**, uma organização secreta, e cada um tem uma **função** no time, com uma habilidade própria. Todo agente entra na partida com uma **pistola**, que é dele e é criada junto com ele (composição). Os agentes já existem antes de serem reunidos em um **esquadrão** (agregação).

Tarefa:

1. **Parte composta:** crie a classe Pistola, cujo constructor recebe e guarda modelo.
2. **Classe base (contrato e composição):** crie a classe AgenteProtocolo, cujo constructor recebe codinome e modeloDaPistola:
   - guarde codinome em um atributo público;
   - crie a Pistola **dentro** do constructor, com o modelo recebido, e guarde-a no atributo privado #pistola;
   - crie o método usarHabilidade(), que apenas lança um erro com throw new Error(...), com uma mensagem à sua escolha;
   - crie o método entrarEmCombate(), que imprime o codinome, o modelo da pistola e o texto **retornado** por usarHabilidade().

EXERCÍCIO 2

3. Crie as classes filhas Controlador, Iniciador e Duelista, sem constructor próprio. Cada uma sobrescreve usarHabilidade() para retornar um texto com uma habilidade típica da sua função, à sua escolha.
4. Agregação: crie a classe Esquadrao, cujo constructor recebe nome e um array de agentes já criados, guardado no atributo privado #agentes, com os métodos:
5. iniciarPartida(), que imprime o nome do esquadrão e, com forEach, chama entrarEmCombate() de cada agente (polimorfismo);
6. listarControladores(), que usa filter com instanceof para retornar um array só com os agentes da classe Controlador.
7. No main, crie dois controladores, um iniciador e um duelista, com codinomes e modelos de pistola à sua escolha. Reúna os quatro em um esquadrão, chame iniciarPartida() e imprima, com forEach, o codinome de cada agente do array retornado por listarControladores().
8. Crie um AgenteProtocolo diretamente e, na linha seguinte, chame entrarEmCombate() nele. Observe que o objeto é criado, mas a chamada gera um erro. Depois, transforme a linha da chamada em comentário (//) para o programa voltar a rodar.

EXERCÍCIO 3

Cenário: Em Gravity Falls, dois irmãos gêmeos passam as férias em uma cidadezinha cheia de mistérios, na casa do tio-avô, que administra a Cabana do Mistério: uma loja de souvenirs e falsas atrações para turistas. Lá, eles encontram um diário misterioso, escrito por um autor desconhecido, que registra as criaturas sobrenaturais da cidade. Cada ficha é escrita dentro do diário e só existe nele (composição). Já os funcionários da Cabana são moradores da cidade, que existiam antes de serem contratados e continuam existindo fora dela (agregação).

Tarefa:

1. Crie a classe FichaDeCriatura, cujo constructor recebe criatura (o nome da criatura) e nivelDePerigo, com o método descrever(), que **retorna** um texto com os dois dados.
2. Crie a classe DiarioMisterioso, cujo constructor recebe numero (o número do diário), com o atributo privado #fichas, que começa como um array vazio, e os métodos:
   - anotar(criatura, nivelDePerigo), que cria uma FichaDeCriatura **dentro** do método e a adiciona ao #fichas com push;
   - exibirFichas(), que imprime o número do diário e, com forEach, o texto retornado por descrever() de cada ficha.

EXERCÍCIO 3

3. Crie a classe Funcionario, cujo constructor recebe nome e cargo.
4. Crie a classe CabanaDoMisterio, cujo constructor recebe dono (o nome do dono), com o atributo privado #funcionarios, que começa como um array vazio, e os métodos:
   - contratar(funcionario), que adiciona um funcionário já criado ao #funcionarios com push;
   - abrirAsPortas(), que imprime o nome do dono e, com forEach, o nome e o cargo de cada funcionário.
5. No main, crie um diário com o número à sua escolha, anote nele duas ou três criaturas à sua escolha e chame exibirFichas(). Depois, crie dois funcionários antes da Cabana, à sua escolha. Por fim, crie a Cabana, contrate os dois e chame abrirAsPortas().
6. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

EXERCÍCIO 4

Cenário: Em Hunter x Hunter, **Hunters** são aventureiros licenciados que dominam o **Nen**, a técnica de controlar a própria energia vital, chamada de **aura**. Todo usuário de Nen pertence a um **tipo de Nen**, que define o seu estilo de luta. A **Trupe Fantasma**, um grupo de ladrões cujos membros são chamados de **Aranhas**, foi avistada em uma cidade, e três Hunters de tipos diferentes vão enfrentá-la.

Tarefa:

1. Crie a classe base UsuarioDeNen, cujo constructor recebe nome, com o método atacar(alvo), que recebe uma Aranha (passo 3) e imprime um ataque genérico de aura contra ela, citando o nome dos dois.
2. Crie as classes filhas Intensificador, Transmutador e Materializador, sem constructor próprio. Cada uma sobrescreve atacar(alvo) para imprimir um ataque típico do seu tipo de Nen, à sua escolha, citando o nome dos dois.
3. Crie a classe Aranha, cujo constructor recebe nome e numero (o número tatuado).

EXERCÍCIO 4

4. Agregação: crie a classe TrupeFantasma, cujo constructor recebe cidade e um array de Aranhas já criadas, com valor padrão [], guardado no atributo privado #aranhas, com os métodos:
   - revelarAranhas(), que imprime a cidade e, com forEach, o número e o nome de cada Aranha;
   - buscarAranha(numero), que usa find para retornar a Aranha com o número informado.
5. No main, crie duas Aranhas, à sua escolha, antes da Trupe. Crie a Trupe com uma cidade à sua escolha, chame revelarAranhas() e use buscarAranha(...) para pegar uma das duas Aranhas.
6. Ainda no main, crie um array com um usuário de Nen de cada tipo (Intensificador, Transmutador e Materializador), com nomes à sua escolha, e percorra-o com forEach, fazendo cada um chamar atacar(...) contra a Aranha encontrada no passo anterior.

FIM DA AULA!

zSh3rl0cK/**S01-monitoria**

